Python logging Module

```
# ============================================================
# LOGGING MODULE – COMPLETE EXPLANATION IN ONE PYTHON SCRIPT
# ============================================================

import logging
from pathlib import Path


# ============================================================
# 1. BASIC LOGGING
# ============================================================

logging.basicConfig(
    level=logging.INFO
)

logging.info("Pipeline started")

print("\nBasic logging completed.")


# ============================================================
# 2. LOGGING LEVELS
# ============================================================

# Logging levels from least to most severe:
#
# DEBUG
# INFO
# WARNING
# ERROR
# CRITICAL


logging.debug("This is a DEBUG message")
logging.info("This is an INFO message")
logging.warning("This is a WARNING message")
logging.error("This is an ERROR message")
logging.critical("This is a CRITICAL message")


# Because our level is INFO:
#
# DEBUG is hidden
# INFO is displayed
# WARNING is displayed
# ERROR is displayed
# CRITICAL is displayed


# ============================================================
# 3. CHANGE LOGGING LEVEL
# ============================================================

# If you want DEBUG messages:

logging.basicConfig(
    level=logging.DEBUG
)

# Note:
# basicConfig() only configures logging once in most scripts.
# We will use a better configuration later.
```

```python
# ============================================================
# 4. LOGGING MESSAGE WITH VARIABLES
# ============================================================

customer = "Ahmed"
records = 100

logging.info(
    "Customer %s processed %d records",
    customer,
    records
)


# ============================================================
# 5. LOGGING DIFFERENT DATA TYPES
# ============================================================

pipeline_name = "customer_etl"
duration = 12.5
success = True

logging.info(
    "Pipeline=%s Duration=%.2f Success=%s",
    pipeline_name,
    duration,
    success
)


# ============================================================
# 6. CREATE A LOG FILE
# ============================================================

log_file = Path("pipeline.log")

logging.basicConfig(
    filename=log_file,
    level=logging.INFO
)

# In a simple standalone script, this configuration
# writes logs to pipeline.log.


# ============================================================
# 7. FORMAT LOG MESSAGES
# ============================================================

# A better production-style configuration:

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message)s"
)

logging.info("Formatted log message")


# Example:
#
# 2026-09-06 03:00:00,123 - INFO - Formatted log message


# ============================================================
# 8. IMPORTANT LOG FORMAT VALUES
# ============================================================

"""
```

```
%(asctime)s
    Date and time

%(levelname)s
    Log level

%(message)s
    Log message

%(name)s
    Logger name

%(filename)s
    Filename

%(lineno)d
    Line number
"""


# ============================================================
# 9. CREATE A LOGGER
# ============================================================

logger = logging.getLogger("etl_pipeline")

logger.setLevel(logging.INFO)

logger.info("ETL logger created")
logger.warning("This is an ETL warning")


# ============================================================
# 10. HANDLING EXCEPTIONS
# ============================================================

try:

    result = 10 / 0

except ZeroDivisionError:

    logger.error(
        "Division by zero occurred"
    )


# ============================================================
# 11. logger.exception()
# ============================================================

try:

    number = int("abc")

except ValueError:

    logger.exception(
        "Failed to convert value to integer"
    )

# logger.exception() automatically includes
# the traceback information.


# ============================================================
# 12. DEBUG LOGGING
# ============================================================
```

```
logger.setLevel(logging.DEBUG)

logger.debug("Starting transformation")
logger.debug("Reading input file")
logger.debug("Applying validation rules")
logger.debug("Writing output file")


# ============================================================
# 13. WARNING EXAMPLE
# ============================================================

age = -5

if age < 0:

    logger.warning(
        "Invalid age detected: %s",
        age
    )


# ============================================================
# 14. ERROR EXAMPLE
# ============================================================

file_path = Path("missing_file.csv")

if not file_path.exists():

    logger.error(
        "File does not exist: %s",
        file_path
    )


# ============================================================
# 15. CRITICAL EXAMPLE
# ============================================================

database_connected = False

if not database_connected:

    logger.critical(
        "Database connection failed!"
    )


# ============================================================
# 16. LOGGER WITH FILE HANDLER
# ============================================================

# A more professional logging configuration.

logger = logging.getLogger("data_pipeline")

logger.setLevel(logging.DEBUG)

# Prevent duplicate handlers if script is run repeatedly
logger.handlers.clear()


# ------------------------------------------------------------
# File Handler
# ------------------------------------------------------------

file_handler = logging.FileHandler(
    "data_pipeline.log",
```

```
    encoding="utf-8"
)

file_handler.setLevel(logging.DEBUG)


# ------------------------------------------------------------
# Console Handler
# ------------------------------------------------------------

console_handler = logging.StreamHandler()

console_handler.setLevel(logging.INFO)


# ------------------------------------------------------------
# Formatter
# ------------------------------------------------------------

formatter = logging.Formatter(
    "%(asctime)s - %(name)s - %(levelname)s - %(message)s"
)

file_handler.setFormatter(formatter)
console_handler.setFormatter(formatter)


# ------------------------------------------------------------
# Add handlers
# ------------------------------------------------------------

logger.addHandler(file_handler)
logger.addHandler(console_handler)


# ============================================================
# 17. TEST THE LOGGER
# ============================================================

logger.debug("Debug information")
logger.info("Pipeline started")
logger.warning("Input contains missing values")
logger.error("One record failed validation")
logger.critical("Critical pipeline failure")


# ============================================================
# 18. REALISTIC ETL EXAMPLE
# ============================================================

logger.info("=" * 50)
logger.info("ETL PIPELINE STARTED")
logger.info("=" * 50)


# ------------------------------------------------------------
# EXTRACT
# ------------------------------------------------------------

logger.info("Starting extraction")

orders = [
    {
        "order_id": 1,
        "customer": "Ahmed",
        "amount": 500
    },
    {
        "order_id": 2,
```

```
        "customer": "Mohamed",
        "amount": 1200
    },
    {
        "order_id": 3,
        "customer": "Omar",
        "amount": -100
    }
]

logger.info(
    "Extracted %d records",
    len(orders)
)


# ------------------------------------------------------------
# TRANSFORM
# ------------------------------------------------------------

logger.info("Starting transformation")

clean_orders = []

for order in orders:

    logger.debug(
        "Processing order %s",
        order["order_id"]
    )

    # Validate amount

    if order["amount"] < 0:

        logger.warning(
            "Invalid amount for order %s: %s",
            order["order_id"],
            order["amount"]
        )

        continue

    # Add status

    if order["amount"] >= 1000:

        order["status"] = "High Value"

    else:

        order["status"] = "Normal"

    clean_orders.append(order)


logger.info(
    "Transformation completed: %d valid records",
    len(clean_orders)
)


# ------------------------------------------------------------
# LOAD
# ------------------------------------------------------------

logger.info("Starting load")

try:
```

```
    output_file = Path("clean_orders.txt")

    with output_file.open(
        "w",
        encoding="utf-8"
    ) as file:

        for order in clean_orders:

            file.write(
                str(order) + "\n"
            )

    logger.info(
        "Successfully loaded %d records to %s",
        len(clean_orders),
        output_file
    )

except Exception:

    logger.exception(
        "Failed to load output data"
    )


# ------------------------------------------------------------
# PIPELINE COMPLETE
# ------------------------------------------------------------

logger.info("=" * 50)
logger.info("ETL PIPELINE COMPLETED")
logger.info("=" * 50)


# ============================================================
# 19. LOGGING PIPELINE STATISTICS
# ============================================================

total_records = len(orders)
valid_records = len(clean_orders)
invalid_records = total_records - valid_records

logger.info(
    "Total records: %d",
    total_records
)

logger.info(
    "Valid records: %d",
    valid_records
)

logger.info(
    "Invalid records: %d",
    invalid_records
)


# ============================================================
# 20. USING LOGGING IN FUNCTIONS
# ============================================================

def extract_data():

    logger.info("Extract function started")

    data = [
```

```
        {"id": 1, "value": 100},
        {"id": 2, "value": 200}
    ]

    logger.info(
        "Extracted %d records",
        len(data)
    )

    return data


def transform_data(data):

    logger.info("Transform function started")

    transformed = []

    for record in data:

        record["value"] *= 2

        transformed.append(record)

    logger.info(
        "Transformed %d records",
        len(transformed)
    )

    return transformed


def load_data(data):

    logger.info("Load function started")

    for record in data:

        logger.debug(
            "Loading record: %s",
            record
        )

    logger.info(
        "Loaded %d records",
        len(data)
    )


# Execute ETL

data = extract_data()

data = transform_data(data)

load_data(data)


# ============================================================
# 21. MAIN FUNCTION
# ============================================================

def main():

    logger.info("Application started")

    try:

        data = extract_data()
```

```
        data = transform_data(data)

        load_data(data)

        logger.info(
            "Application completed successfully"
        )

    except Exception:

        logger.exception(
            "Application failed"
        )


if __name__ == "__main__":

    main()


# ============================================================
# 22. BASIC print() vs logging
# ============================================================

"""
print():

    print("Pipeline started")


logging:

    logger.info("Pipeline started")


Why logging is better for production:

    print()
        ↓
    Simple output


    logging
        ↓
    Timestamp
    ↓
    Level
    ↓
    Logger name
    ↓
    File
    ↓
    Console
    ↓
    Error traceback
"""


# ============================================================
# 23. LOGGING LEVELS SUMMARY
# ============================================================

print("""
============================================================
LOGGING LEVELS
============================================================

DEBUG
```

```
    Detailed information for debugging.

INFO
    Normal application/pipeline progress.

WARNING
    Something unexpected but the pipeline can continue.

ERROR
    An operation failed.

CRITICAL
    A very serious failure.


============================================================
IMPORTANT FUNCTIONS
============================================================

logging.debug()
logging.info()
logging.warning()
logging.error()
logging.critical()

logger.debug()
logger.info()
logger.warning()
logger.error()
logger.critical()

logger.exception()
    Log an exception with traceback.


============================================================
IMPORTANT CLASSES
============================================================

logging.getLogger()
    Create/get a logger.

logging.FileHandler()
    Write logs to a file.

logging.StreamHandler()
    Write logs to console.

logging.Formatter()
    Control log format.


============================================================
DATA ENGINEERING USE
============================================================

ETL Pipeline
     ↓
    LOG
     ↓
Extract started
     ↓
1000 records extracted
     ↓
Transformation started
     ↓
20 invalid records
     ↓
980 records transformed
```

```
     ↓
Load started
     ↓
980 records loaded
     ↓
Pipeline completed


============================================================
KEY IDEA
============================================================

print()
    → Good for quick debugging.

logging
    → Good for real applications and production pipelines.
============================================================
""")
```